

Tema: Investigar y desarrollar una aplicación de consola en .NET que permita la deserialización de un archivo JSON.

Lineamientos de la aplicación:

- El archivo a deserializar se encuentra en la carpeta Material Complementario con el nombre de weatherV1.json
- Utilice la API System.Text.Json para deserializar los datos JSON en objetos .NET
- Imprime las propiedades del objeto en consola

Para este trabajo donde nos solicitan imprimir las propiedades de un objeto en consola, utilizaremos la tecnología .net core, dado a que es una herramienta multiplataforma, es ideal para desarrollos y ejecuciones no solamente en Windows, sino también en MacOS y Linux.

Luego de establecer la tecnología que vamos a utilizar, crearemos la carpeta del proyecto, en este caso será desde un terminal del ordenador bajo el siguiente comando:

> dotnet new console -n WeatherV1.json

Luego ingresaremos a la carpeta que acabamos de crear desde nuestra consola

```
C:\>WeatherV1.json
"WeatherV1.json" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

C:\>WeatherV1.json\WeatherV1.json.csproj

C:\>
[main 2026-05-06T04:13:33.134Z] StorageMainService: creating application shared storage
[main 2026-05-06T04:13:33.135Z] StorageMainService: creating application shared storage with local application storage fallback
[main 2026-05-06T04:13:33.138Z] CrossAppIPCSERVICE: connecting to peer
[main 2026-05-06T04:13:33.236Z] [shared storage] Creating shared storage database at 'c:\Users\Aurelio Reyes\.vscode-shared\sharedStorage\state.vscdb' (wasCreated: false)
[main 2026-05-06T04:13:33.237Z] [shared storage] Initializing fallback application storage (type: local, path: c:\Users\Aurelio Reyes\AppData\Roaming\Code\User\globalStorage\state.vscdb)
[main 2026-05-06T04:13:33.282Z] update#setState disabled
[main 2026-05-06T04:13:33.283Z] update#ctor - updates are disabled due to running as Admin in user setup
[main 2026-05-06T04:13:33.289Z] [shared storage] Fallback application storage initialized with 112 items
[main 2026-05-06T04:14:38.900Z] Extension host with pid 25912 exited with code: 0, signal: unknown.
```

Ya en este punto definiremos primeramente nuestra estructura del proyecto, la cual esta asi:

C:\ WeatherV1.json

|-- WeatherV1.json.csproj

|--Program.cs

|--Material Complementario/

|-- weatherV1.json

Y ya en este punto podremos trabajar en nuestro proyecto.

Ejecución de sentencias y Código fuentes.

WeatherV1.json.csproj

El archivo csproj es el archivo principal del proyecto, es un archivo XML que indica al motor de .NET como se debe compilar.

Explicación Del Código WeatherV1.json.csproj:

`<Project Sdk="Microsoft.NET.Sdk">` //esta línea que se está usando Software Development Kit para .NET

`<PropertyGroup>`

`<OutputType>Exe</OutputType>` //Define que nuestro resultado final sea ejecutable

`<TargetFramework>net10.0</TargetFramework>` //Se indica que versión de .NET se esta usando

`<ImplicitUsings>enable</ImplicitUsings>` //Permite que nuestro compilador sepa algunos using.

`<Nullable>enable</Nullable>` //Activa los rastreos nulos

`</PropertyGroup>`

`<ItemGroup>`

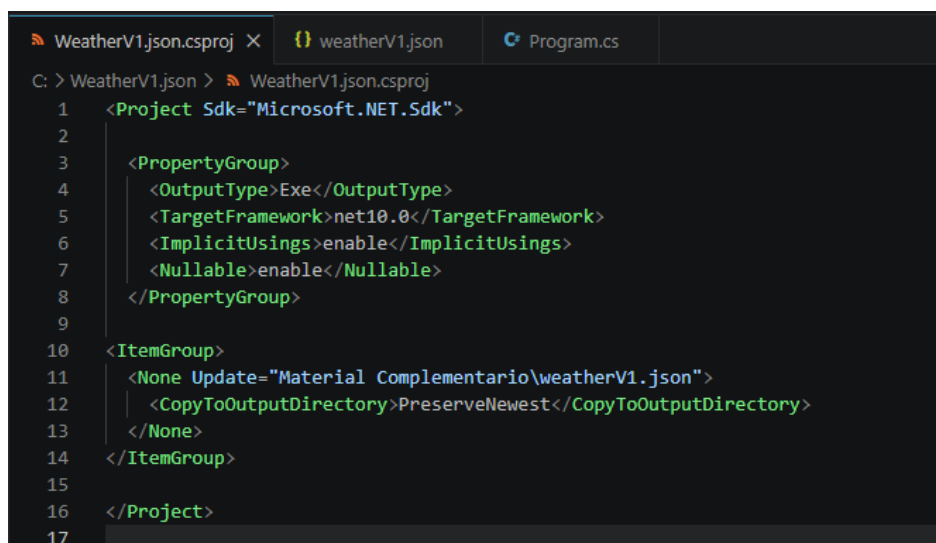
`<None Update="Material Complementario\weatherV1.json">` //Se indica que el archivo es importante pero que no se debe compilar

`<CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>` //Instrucción ante errores de archivos no encontrados

`</None>`

`</ItemGroup>`

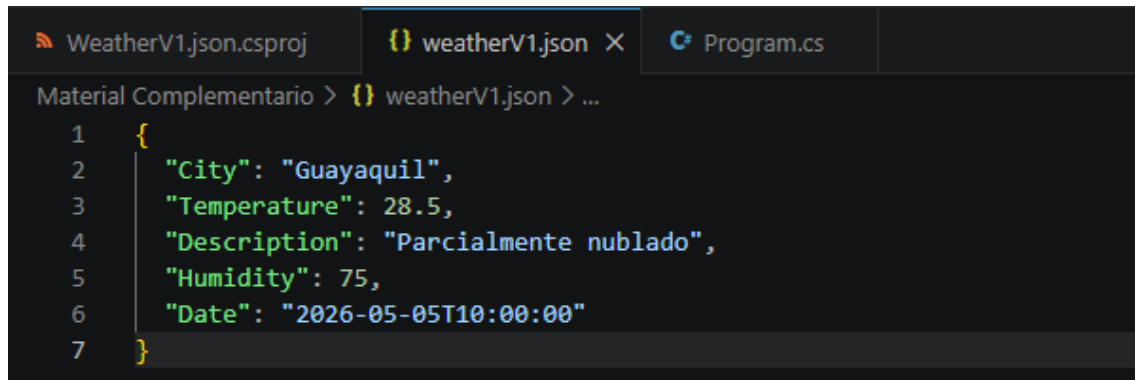
`</Project>`



```
1 <Project Sdk="Microsoft.NET.Sdk">
2
3   <PropertyGroup>
4     <OutputType>Exe</OutputType>
5     <TargetFramework>net10.0</TargetFramework>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9
10  <ItemGroup>
11    <None Update="Material Complementario\weatherV1.json">
12      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
13    </None>
14  </ItemGroup>
15
16 </Project>
17
```

WeatherV1.json.

Este archivo no es un código de programación, sino un formato de intercambio de datos, el cual cumple la función de organizar la información que sea entendible tanto para el usuario como para la máquina.



```
WeatherV1.json.csproj  {} weatherV1.json X  Program.cs
Material Complementario > {} weatherV1.json > ...
1  {
2    "City": "Guayaquil",
3    "Temperature": 28.5,
4    "Description": "Parcialmente nublado",
5    "Humidity": 75,
6    "Date": "2026-05-05T10:00:00"
7  }
```

Programs.cs

Este archivo es el corazón de nuestra aplicación y es la encargada de transformar nuestro archivo de texto (WeatherV1.json.) en un objeto de programación con el que podemos trabajar.

Explicación Del Código WeatherV1.json.csproj:

```
using System;
using System.IO;
using System.Text.Json;
namespace WeatherApp
{
    // Definición del objeto .NET para poder mapear al JSON
    public class WeatherData
    {
        public string City { get; set; } = string.Empty;
        public double Temperature { get; set; }
        public string Description { get; set; } = string.Empty;
        public int Humidity { get; set; }
        public DateTime Date { get; set; }
    }
    class Program
```

```
{
    static void Main(string[] args)
    {
        // Carpeta 'Material Complementario'
        string filePath = Path.Combine("Material Complementario", "weatherV1.json");
        try
        {
            // Leer archivo
            string jsonString = File.ReadAllText(filePath);
            // API System.Text.Json
            var options = new JsonSerializerOptions { PropertyNameCaseInsensitive =
true };
            WeatherData? weather = JsonSerializer.Deserialize<WeatherData>(jsonString,
options);
            // impresión de las propiedades del objeto en consola
            if (weather != null)
            {
                Console.WriteLine("Propiedades del objeto deserializado:");
                Console.WriteLine("-----");
                Console.WriteLine($"Ciudad:    {weather.City}");
                Console.WriteLine($"Temperatura: {weather.Temperature} °C");
                Console.WriteLine($"Descripción: {weather.Description}");
                Console.WriteLine($"Humedad:    {weather.Humidity}%");
                Console.WriteLine($"Fecha:     {weather.Date}");
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error al procesar el archivo: {ex.Message}");
        }
    }
}
```

```
}  
  
}
```

```
WeatherV1.json.csproj weatherV1.json Program.cs X  
C: > WeatherV1.json > Program.cs > ...  
1 using System;  
2 using System.IO;  
3 using System.Text.Json;  
4 namespace WeatherApp  
5 {  
6     // Definición del objeto .NET para poder mapear al JSON  
7     2 references  
8     public class WeatherData  
9     {  
10         1 reference  
11         public string City { get; set; } = string.Empty;  
12         1 reference  
13         public double Temperature { get; set; }  
14         1 reference  
15         public string Description { get; set; } = string.Empty;  
16         1 reference  
17         public int Humidity { get; set; }  
18         1 reference  
19         public DateTime Date { get; set; }  
20     }  
21     0 references  
22     class Program  
23     {  
24         0 references  
25         static void Main(string[] args)  
26         {  
27             // Carpeta 'Material Complementario'  
28             string filePath = Path.Combine("Material Complementario", "weatherV1.json");  
29             try  
30             {  
31                 // Leer archivo  
32                 string jsonString = File.ReadAllText(filePath);  
33                 // API System.Text.Json  
34                 var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };  
35                 WeatherData? weather = JsonSerializer.Deserialize<WeatherData>(jsonString, options);  
36                 // impresión de las propiedades del objeto en consola  
37                 if (weather != null)  
38                 {  
39                     Console.WriteLine("Propiedades del objeto deserializado:");  
40                     Console.WriteLine("-----");  
41                     Console.WriteLine($"Ciudad: {weather.City}");  
42                     Console.WriteLine($"Temperatura: {weather.Temperature} °C");  
43                     Console.WriteLine($"Descripción: {weather.Description}");  
44                     Console.WriteLine($"Humedad: {weather.Humidity}%");  
45                     Console.WriteLine($"Fecha: {weather.Date}");  
46                 }  
47             }  
48             catch (Exception ex)
```

Ejecución de programa

Utilizamos el comando > **dotnet run** desde la ruta donde se encuentra nuestro archivo .csproj

Resultado//

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\WeatherV1.json> dotnet run  
"required" or declare the property as a value that accepts null values.  
C:\WeatherV1.json\Program.cs(15,27): warning CS8600: Se va a convertir un literal nulo o un posible valor nulo en un tipo que no acepta valores NULL.  
C:\WeatherV1.json\Program.cs(19,39): warning CS8602: Desreferencia de una referencia posiblemente NULL.  
--- DATOS DEL CLIMA DESERIALIZADOS ---  
Ciudad: Guayaquil  
Temperatura: 28,5°C  
Descripción: Parcialmente nublado  
Humedad: 75%  
Fecha: 5/5/2026 10:00:00  
PS C:\WeatherV1.json> dotnet run  
Propiedades del objeto deserializado:  
-----  
Ciudad: Guayaquil  
Temperatura: 28,5 °C  
Descripción: Parcialmente nublado  
Humedad: 75%  
Fecha: 5/5/2026 10:00:00  
PS C:\WeatherV1.json> []
```